

Practical 4. Optimal fitting and forecasting

In this practice, we show how to choose the **best ARMA model** and how to forecast with these family of models.

1. Choose p and q and simulate an ARMA (p,q) model

- `x<-arima.sim(list(order=c(p,0,q), ar=c(a1, a2,...,ap), ma=c(b1,...,bq)), n)`
- `plot (x, type="l")`

Type in:

- `library(forecast)`
- `y<-auto.arima (x)`
- `y`

Observe the result.

Function `auto.arima` is in the package “forecast”.

Try

- `y<-auto.arima (x, max.p=2, max.d=0,max.q=2)`

Compute

- `z<-resid(y)`

Check that z is an IID noise.

To forecast we use the instructions

- `xfc<-forecast(x)`
- `plot (xfc, type="l")`

Example:

- `n<-1000`
- `x<-arima.sim(list(order=c(1,0,1), ar=c(0.8), ma=c(0.3)), n)`
- `plot(x, type="l")`
- `Box.test(x, lag=h, type=c("Ljung-Box"))`
- `library(forecast)`
- `y<-auto.arima(x)`
- `y`
- `z<-resid(y)`
- `plot(z)`
- `h<-log(n)`
- `Box.test(z, lag=h, type=c("Ljung-Box"))`
- `xfc<-forecast(x)`
- `plot(xfc, type="l")`

Do the same, now with real data.

- `y<-LakeHuron`
- `plot(y)`
- `lh<-auto.arima(y)`
- `lh`
- `z<-resid(lh)`

Check that z is an IID noise.

To forecast do

- `lhfc<-forecast(lh)`
- `plot(lhfc, type="l")`

Example:

- `x<-LakeHuron`
- `plot(x,type="l")`
- `lh<-auto.arima(x)`
- `lh`
- `z<-resid(lh)`
- `plot(z,type="l")`
- `h<-log(98)`
- `Box.test(z,lag=h, type=c("Ljung-Box"))`
- `lhfc<-forecast(lh)`
- `plot(lhfc, type="l")`

Next example improves the randomness of the residuals:

- `x<-LakeHuron`
- `plot(x,type="l")`
- `lh<-arima(x, order=c(1,1,1), method="ML")`
- `lh`
- `z<-resid(lh)`
- `plot(z,type="l")`
- `h<-log(98)`
- `Box.test(z,lag=h, type=c("Ljung-Box"))`
- `lhfc<-forecast(lh)`
- `plot(lhfc, type="l")`

Another version:

- `x<-LakeHuron`
- `y<-diff(x)`
- `plot(y,type="l")`
- `lh<-arima(y, order=c(1,0,1), method="ML")`
- `lh`
- `z<-resid(lh)`
- `plot(z,type="l")`
- `h<-log(98)`
- `Box.test(z,lag=h, type=c("Ljung-Box"))`
- `library(forecast)`
- `lhfc<-forecast(lh)`
- `plot(lhfc, type="l")`

Try

- `x<-LakeHuron`
- `y<-diff(x)`
- `plot(y,type="l")`
- `lh<-arima(y, order=c(2,0,2), method="ML")`
- `lh`
- `z<-resid(lh)`
- `plot(z,type="l")`
- `h<-log(98)`
- `Box.test(z,lag=h, type=c("Ljung-Box"))`
- `library(forecast)`
- `lhfc<-forecast(lh)`
- `plot(lhfc, type="l")`

Analyze what happens

2. Generate an MA (2) and fit an MA (3), for example. Note that the third coefficient is statistically nul. Note that it is better to force mean=0 to avoid “statistical noise”. Try the following:

- `n<-1000`
- `x<-arima.sim(list(order=c(0,0,2), ma=c(1.5,0.75)),n)`
- `plot(x,type="l")`
- `x.ma2<-arima(x,order=c(0,0,2),include.mean=F)`
- `x.ma2`
- `y2<-resid(x.ma2)`
- `plot(y2,type="l")`
- `Box.test(y2, lag = log(n), type = c("Ljung-Box"))`
- `x.ma3<-arima(x,order=c(0,0,3),include.mean=F)`
- `x.ma3`
- `y3<-resid(x.ma3)`
- `plot(y3,type="l")`
- `Box.test(y3, lag = log(n), type = c("Ljung-Box"))`

Play with other parameters, number of observations, etc.

3. Repeat the previous exercise with an AR model.

4. Sometimes it may be difficult to distinguish more complex ARMA models like an ARMA (2,1) and an ARMA (1,2). Thus, the fitting is frequently not that good. Fit an ARIMA (a, 1, b) model to Lake Huron data.

- `x<-LakeHuron`
- `y<-diff(x)`
- `plot(y,type="l")`
- `a<-2`
- `b<-2`
- `lh<-arima(y,order=c(a,0,b), method="ML", include.mean=F)`
- `lh`
- `z<-resid(lh)`
- `plot(z,type="l")`
- `h<-5`
- `Box.test(z,lag=h,type=c("Ljung-Box"))`

- Check that z is an IID noise white noise. It should be clear.
- Move h and see what happens. Note that $\log(98)$ is close to 5.
- Decrease a and b and see what happens. When a and b decrease the noise character of z becomes less clear; note that the p-value decreases.
- In this case, it seems that `auto.arima`, that says that y is an IID noise, is not good enough. Probably 98 data is insufficient.

5. Do

- `library(help = "datasets")`

In the package `datasets` you can find many time series. Choose some of them and fit the best ARIMA model. Recall that if x is an ARIMA (p, 1, q) model, its first differences is an ARMA (p, q) model. We will see this in Lecture 11.

Practical 11. Dickey-Fuller test and SARIMA models

A) The Dickey-Fuller test:

Dickey-Fuller test allows to distinguish between a random walk and an AR(1) model when parameter ϕ is close to 1. The null hypothesis is $\phi=1$. Simulate a white noise:

- `n<-200`
- `x<-rnorm(n)`
- `x`

Download the package “tseries”.

Then, do

- `adf.test(x)`

and

- `pp.test(x).`

Note that the p-value is very small so we reject the hypothesis of random walk.

Do now

- `y<-cumsum(x)`
- `y`
- `ts.plot(y)` (or `plot(y,type="l")`)

This is a random walk. Do

- `adf.test(y)`
- `pp.test(y)`

Now, the p-value is large and, therefore, we accept the hypothesis that y is a random walk, or equivalently, $\phi=1$. Therefore, it may be convenient to take first differences to adjust an ARMA model. Simulate an AR(1) model with $\phi=0.9$. Check what happens. Compare different values of ϕ , 0.9, 0.99,... and different values of n . For example try first $n=1000$ and $\phi=0.9$. Then, $\phi=0.99$. And then $n=10,000$.

B) Fitting SARIMA models:

Data in “nottem” are monthly temperatures at Nottingham Castle in Fahrenheit degrees from 1921 to 1940.

Note that we fit a SARIMA(1,0,1)(1,1,1)₁₂.

- `x<-nottem`
- `plot(x)`
- `y<-diff(x,12)`
- `plot(y)`
- `par(mfrow=c(2,1))`
- `acf(y, 48)`
- `pacf(y, 48)`
- `fit1<-arima(y, order=c(1,0,1), seasonal=list(order=c(1,0,1),period=12))`
- `fit1`
- `fit2=resid(fit1)`
- `fit2`
- `plot(fit2)`
- `par(mfrow=c(2,1))`
- `acf(fit2, 48)`
- `pacf(fit2, 48)`
- `predict(fit1,n.ahead=24)`

- `library(forecast)`
- `par(mfrow=c(1,1))`
- `fit1<-forecast(fit1)`
- `plot(fit1 for)`

Try to use less parameters maintaining the IID noised character of the residuals.
Another example is the following

- `x<-AirPassengers`
- `plot(x)`
- `library(astsa)`
- `fit1<-sarima(x,0,1,0,0,1,0,12)`
- `fit2<-sarima(x,1,1,0,0,1,0,12)`
- `fit3<-sarima(x,0,1,0,0,1,1,12)`

Compare the different fittings. What is the best model?

To make a prediction, do for example

```
sarima.for(x,12,0,1,0,0,1,1,12)
```